Simulador de Asignación de Memoria y Planificación de Procesos

Sistemas Operativos — UTN FRRe — 2026

Guía del grupo: tareas, diagrama corregido y explicación de cada módulo

1. Presentación del proyecto
2. HOWTO: instalación y uso
3. Tareas por miembro
4. Diagrama corregido
5. M1 — Carga
6. M2 — Proceso
7. M3 — Memoria
8. M4 — SRTF
9. M5 — Admisión
10. M6 — Simulador
11. M7 — Salida

Simulador de Asignación de Memoria y Planificación de Procesos

TPI de Sistemas Operativos — UTN FRRe — 2026

- **Memoria:** particiones variables (MVT), 100K para el SO + 450K para usuarios, política **Best-Fit**
- **CPU:** un solo procesador, **SRTF** (apropiativo)
- **Grado de multiprogramación:** máximo 5 (Ejecución + Listo + Listo/Suspendido)
- **Estados:** Nuevo, Listo, Listo/Suspendido, Ejecución, Terminado
- **Lenguaje:** Python 3.10 o superior, **sin librerías externas**
- **Entrega:** instalador `SimuladorSetup.exe` que funciona sin Python y sin internet

Cómo ejecutarlo (desarrollo)

```
python main.py datos/procesos-ejemplo.csv
```

```
python -m unittest discover -s tests -v
```

Sin archivo (`python main.py`), se cargan los procesos a mano por teclado.

Estructura

```
main.py                     entrada por consola                          (M6)
simulador/
  config.py                 parámetros de la consigna (no hardcodear números)
  carga.py                  leer CSV, validar, carga manual               (M1)
  proceso.py                clase Proceso (PCB) y estados                 (M2)
  memoria.py                MVT + Best-Fit                                (M3)
  planificador.py           SRTF + diagrama de Gantt                      (M4)
  admision.py               largo y mediano plazo, límite de 5            (M5)
  motor.py                  reloj y ciclo de simulación                   (M6)
  estadisticas.py, salida.py  informe y pantallas                         (M7)
tests/                      un archivo de tests por módulo
datos/                      archivos CSV de prueba
instalador/                 construir.bat + script de Inno Setup          (M7)
HOWTO.md                    instrucciones para el profe
docs/
  TAREAS-TRELLO.md          tareas por miembro y sprints
  diagrama-corregido.md     errores del diagrama original + versión corregida
  modulos/                  guía de cada módulo: qué es, por qué y cómo exponerlo
```

Cómo trabajamos

1. Cada miembro programa **sólo el archivo de su módulo**, donde dice `TODO Mx`.
2. Su módulo está terminado cuando pasan sus tests, por ejemplo: `python -m unittest tests.test_memoria -v`.

3. **No cambiar los nombres de funciones ni de campos** (son el contrato entre módulos). Si hace falta cambiarlos, se avisa en el grupo.
4. Al terminar, completar la sección "Cómo lo implementé" de su guía en `docs/modulos/`.

HOWTO — Cómo instalar y usar el simulador

> Borrador: lo completa el Miembro 7 cuando el instalador esté probado.

Instalación (no requiere Python ni internet)

1. Ejecutar `SimuladorSetup.exe`.
2. Siguiente → Siguiente → Instalar. No pide permisos de administrador.
3. Queda en el menú Inicio: **Simulador (ejemplo)** y **Simulador (carga manual)**.

Uso

- **Con archivo**: `simulador.exe ruta\al\archivo.csv`
- **Carga manual**: abrir `simulador.exe` sin parámetros y responder lo que pide.
- En cada evento (llega o termina un proceso) la simulación se detiene: presionar **Enter** para continuar.

Formato del archivo

```
id,tamano,arribo,irrupcion
1,200,0,5
2,150,1,2
```

Máximo 10 procesos. El tamaño se expresa en K y no puede superar 450.

Desde el código fuente (para desarrollo)

Requiere Python 3.10 o superior.

```
python main.py datos\procesos-ejemplo.csv
python -m unittest discover -s tests -v
```

Tareas para Trello — Simulador de Memoria y Procesos

Entrega final: 17/11. Quedan unas 3 semanas, así que los sprints son cortos.

Listas sugeridas en el tablero

`Backlog` · `Sprint actual` · `En progreso` · `En revisión` · `Hecho`

Etiquetas: `M1` … `M7` (por miembro), `código`, `tests`, `docs`, `video`, `bloqueante`.

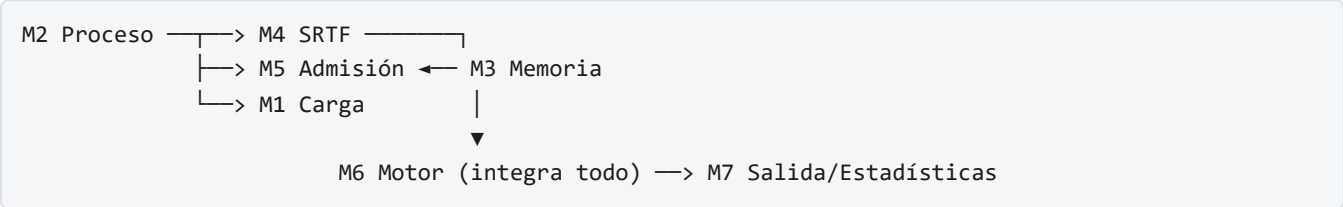
Tarjeta 0 — Preparar el entorno (todos, el primer día)

- [ ] Instalar **Python 3.12** desde python.org (marcar **"Add python.exe to PATH"** en el instalador)
- [ ] Instalar VS Code con la extensión **Python**
- [ ] Copiar la carpeta del proyecto (o clonar el repo, si lo usamos)
- [ ] Correr `python -m unittest discover -s tests -v`: tiene que ejecutar los tests (fallan porque falta el código; eso está bien)

Regla de "Hecho" (vale para todas las tarjetas de código)

1. Pasan los tests de tu módulo: `python -m unittest tests.test_<tu_modulo> -v`
2. No cambiaste el **contrato** (nombres de funciones y campos) sin avisar en el grupo.
3. Completaste la sección "Cómo lo implementé" de tu guía en `docs/modulos/`.
4. Otra persona del grupo lo revisó (pasa por "En revisión").

Orden de dependencias



M2 y M3 son las bases: si alguien se traba, se le da prioridad a esos dos. **M2 tiene que terminar la clase `Proceso` en los primeros 2 días.** El resto puede avanzar en paralelo usando los tests, que no necesitan que los otros módulos estén listos (salvo M5, que usa la memoria de M3, y M6, que integra todo).

Sprint 1 — Módulos en paralelo (27/10 → 03/11)

🟦 M1 — Carga y validación de procesos

Archivo: `simulador/carga.py` · **Tests:** `tests/test_carga.py` · **Guía:** `docs/modulos/01-carga.md`

- [ ] `parsear_csv`: lee el CSV con encabezado; ignora líneas vacías y espacios; soporta `\r\n`
- [ ] Error con número de línea si una fila está mal

- [ ] `validar_procesos`: máx. 10, ids únicos, tamaño 1..450, arribo ≥ 0, irrupción ≥ 1, enteros; junta TODOS los errores
- [ ] `cargar_archivo`: lee, parsea, valida y devuelve `Proceso[]`
- [ ] `carga_manual`: pedir los datos por teclado (re-preguntar si un dato es inválido)
- [ ] Opcional: guardar la carga manual como CSV
- [ ] Crear 2 archivos de prueba con errores en `datos/` para la demo

🟩 M2 — Modelo de Proceso, estados y casos de prueba

Archivo: `simulador/proceso.py` · **Tests:** `tests/test_proceso.py` · **Guía:** `docs/modulos/02-proceso.md`

- [ ] **(Día 1-2, bloqueante)** Validar transiciones en `cambiar_estado` (Nuevo → Ejecución tiene que fallar)
- [ ] Propiedades `tiempo_retorno` y `tiempo_espera`
- [ ] Diagrama de estados del proceso (para el video)
- [ ] Diseñar **3 casos de prueba** en `datos/` y resolverlos **a mano** (Gantt + tabla de memoria por evento + TR/TE): uno con fragmentación externa, uno con el límite de 5 procesos y uno con CPU ociosa
- [ ] Pasar esos 3 casos a `tests/test_integracion.py` (junto con M6)

🟧 M3 — Memoria MVT con Best-Fit

Archivo: `simulador/memoria.py` · **Tests:** `tests/test_memoria.py` · **Guía:** `docs/modulos/03-memoria.md`

- [ ] `buscar_hueco_best_fit` (hueco más chico que alcance; si empatan, el de menor dirección)
- [ ] `asignar`: partir el hueco en partición exacta + resto libre
- [ ] `liberar`: marcar libre y **fusionar** con los huecos contiguos
- [ ] `tabla_particiones` con fragmentación interna
- [ ] `memoria_libre_total`
- [ ] Dibujo de la memoria antes y después de asignar y de liberar (para el video)

🟥 M4 — Planificador SRTF y diagrama de Gantt

Archivo: `simulador/planificador.py` · **Tests:** `tests/test_planificador.py` · **Guía:** `docs/modulos/04-srtf.md`

- [ ] `clave_srtf` con desempates (arribo, id)
- [ ] `elegir_siguiente` (sólo Listos; None si no hay)
- [ ] `planificar`: asignar CPU libre y apropiación **estricta**
- [ ] `Gantt.registrar`, `tramos`, `a_texto`
- [ ] Ejemplo de SRTF vs SJF en una filmina (por qué SRTF es apropiativo)

🟥 M5 — Admisión: largo y mediano plazo

Archivo: `simulador/admision.py` · **Tests:** `tests/test_admision.py` · **Guía:** `docs/modulos/05-admision.md`

- [ ] `contar_en_sistema` (Ejec + Listo + Listo/Susp)
- [ ] `admitir_nuevos`: `arribo <= t`, por orden de arribo; respeta el límite de 5; Listo o Listo/Susp según la memoria
- [ ] `promover_suspendidos`: por orden de arribo, sin bloquearse si uno no entra

- [ ] Mientras M3 no termine, probar con una memoria "falsa" (un objeto con `asignar` que devuelve true/false)
- [ ] Explicación de los 3 planificadores (largo, mediano y corto plazo) para el video

🟪 M6 — Motor de simulación e integración *(sugerido: líder)*

Archivos: `simulador/motor.py`, `main.py` · **Tests:** `tests/test_integracion.py` · **Guía:** `docs/modulos/06-simulador.md`

- [ ] `finalizada`, `paso`, `instantanea` respetando el **orden del ciclo** del diagrama corregido
- [ ] Detectar eventos (arribo / terminación) → `hay_evento`
- [ ] Pausa con Enter en cada evento (consigna: nada de corridas ininterrumpidas)
- [ ] Integrar los módulos a medida que se terminan; correr todos los tests: `python -m unittest discover -s tests -v`
- [ ] Mantener actualizado `docs/diagrama-corregido.md`

🟪 M7 — Salida, estadísticas y entrega

Archivos: `simulador/salida.py`, `simulador/estadisticas.py`, `instalador/` · **Tests:** `tests/test_salida.py` · **Guía:** `docs/modulos/07-salida.md`

- [ ] `calcular_estadisticas`: TR, TE, promedios y rendimiento
- [ ] `render_instante`: CPU, tabla de particiones, colas y nuevos en espera (tablas alineadas)
- [ ] `render_informe`: tabla final con 2 decimales
- [ ] Generar `simulador.exe` con **PyInstaller** y el instalador `SimuladorSetup.exe` con **Inno Setup** (`instalador/construir.bat`)
- [ ] Probar el instalador en una PC **sin Python** y sin internet
- [ ] Completar `HOWTO.md` (lo pide la entrega)

Sprint 2 — Integración y pruebas (03/11 → 10/11)

Tarjeta	Responsable
Integrar todos los módulos y que todos los tests den 0 fallas	M6
Correr los 3 casos a mano de M2 en el simulador y comparar	M2 + M6
Pruebas "rompiendo" el programa: archivos mal formados, 10 procesos, procesos de 450K	M1 + M5
Revisión cruzada de código: cada uno revisa el módulo del siguiente (M1→M2, ..., M7→M1)	Todos
Instalador + HOWTO probado en una PC sin Python	M7
(Opcional) Interfaz gráfica con los estilos: Tkinter o pywebview	A definir

Sprint 3 — Video y entrega (10/11 → 17/11)

Tarjeta	Responsable
Guion del video (cada miembro explica su módulo, 1-2 min)	Líder
Grabar la demo con `datos/procesos-ejemplo.csv`	M6
Cada uno graba su parte siguiendo la sección "Para exponer" de su guía	Todos
Armar el paquete: SimuladorSetup.exe + código fuente + HOWTO → campus	M7 + Líder
Compartir el tablero de Trello con los docentes	Líder

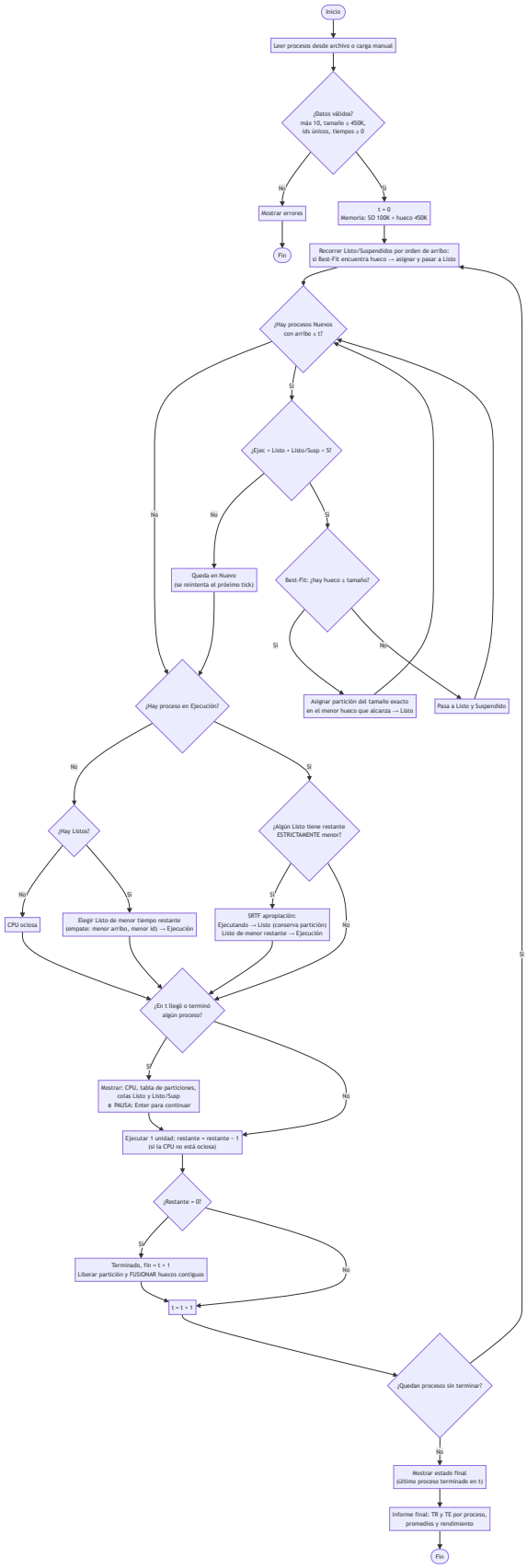
Diagrama del simulador — revisión y versión corregida

Errores encontrados en el diagrama original

#	Dónde	Problema	Corrección
1	`Inicio` → `Avanzar reloj`	Se hace `t = t + 1` **antes** de mirar los arribos, así que un proceso con `arribo = 0` nunca se detecta.	El reloj avanza **al final** de cada ciclo, después de ejecutar.
2	`¿Llegó proceso Nuevo?` `tiempo_arribo = t`	Con `=` un proceso que llegó pero **no entró por falta de cupo** se pierde para siempre: en el tick siguiente `arribo ≠ t`.	Usar `arribo <= t` **y** `estado == Nuevo` (cola de Nuevos que se reintenta cada tick).
3	Mismo rombo	Trata **un** proceso por tick. Pueden llegar varios en el mismo instante.	Recorrer **todos** los nuevos pendientes, ordenados por arribo (y por id si empatan).
4	Orden admisión ↔ suspendidos	Se admiten nuevos **antes** de subir los Listo/Suspendidos. Un proceso recién llegado puede quedarse con el hueco que esperaba uno que llegó antes (injusto, puede generar inanición).	Primero intentar cargar a memoria los **Listo/Suspendidos** (más viejos), después admitir Nuevos.
5	`¿Hay proceso en Ejecución?` → **No** → `Elegir de Listos`	Si la cola de Listos está vacía no hay a quién elegir, y luego se "descuenta 1" a un proceso inexistente.	Agregar `¿Hay Listos?` → No → **CPU ociosa** (no se descuenta nada).
6	Nodo de apropiación SRTF	Dice `Nuevo -> Ejecución`. **Un proceso Nuevo nunca pasa directo a Ejecución**: primero debe tener memoria y estar en Listo.	`Ejecutando -> Listo (conserva partición)` y `Listo de menor restante -> Ejecución`.
7	Comparación SRTF	No define el **empate** (igual tiempo restante).	Sólo se apropia si es **estrictamente menor**. Empate entre Listos: menor arribo, luego menor id.
8	`Proceso Terminado: libera partición`	En MVT al liberar hay que **fusionar huecos libres contiguos**; si no, la memoria queda partida en pedacitos que no sirven.	`Liberar partición y fusionar con huecos adyacentes`. Registrar `fin = t + 1`.
9	`Mostrar salida`	Se muestra **en cada tick** y sin pausa. La consigna pide mostrar cuando **llega** o **termina** un proceso y **prohíbe corridas ininterrumpidas**.	`¿Hubo evento (arribo o fin) en t?` → Sí → mostrar + **pausa (Enter)**.
10	Falta	No hay validación de la carga: máx. 10 procesos, tamaño ≤ 450K, ids únicos, tiempos válidos. Un proceso de 500K nunca entraría y el simulador no termina nunca.	Paso `Validar procesos` después de leer el archivo.
11	Concepto (tabla de particiones)	La consigna pide "fragmentación interna", pero con **particiones variables (MVT)** la partición se crea del tamaño exacto del proceso → la fragmentación interna es **siempre 0**. Lo que aparece en MVT es fragmentación **externa** (huecos sueltos).	Mostrar la columna en 0 y saber explicarlo en el coloquio (es una pregunta típica). Opcional: mostrar también la fragmentación externa.

Diagrama corregido

Se puede ver en VS Code (extensión *Markdown Preview Mermaid*), en GitHub o pegándolo en https://mermaid.live



Convención de tiempo (importante para que todos calculen igual)

- Un "tick" `t` representa el intervalo `[t, t+1)`.
- En el instante `t`: primero se suben suspendidos, después se admiten nuevos, después decide SRTF, después se muestra (si hubo evento) y por último se ejecuta 1 unidad.
- Un proceso que ejecuta su última unidad en el tick `t` **termina en el instante** `t + 1` → `fin = t + 1`.
- La terminación es un evento del instante `t + 1`, así que se muestra al comienzo del ciclo siguiente (ya con la memoria liberada y el nuevo proceso elegido).
- **Tiempo de retorno** `TR = fin − arribo`. **Tiempo de espera** `TE = TR − irrupción`.
- **Rendimiento** = procesos terminados / tiempo total de simulación (`t` final).

M1 — Carga y validación de procesos

Qué es

Es la puerta de entrada del simulador. Lee los procesos desde un archivo CSV (o por teclado), revisa que los datos tengan sentido y crea los objetos `Proceso` que usa el resto del programa.

Formato del archivo (`datos/procesos-ejemplo.csv`):

```
id,tamano,arribo,irrupcion
1,200,0,5
```

- **tamano**: en K, lo que ocupa el proceso en memoria.
- **arribo**: instante en que llega al sistema.
- **irrupción** (*burst*): cuánto tiempo de CPU necesita en total.

Por qué se hizo así

- **CSV**: la consigna dice "leer desde un archivo". CSV se edita con el Bloc de notas o con Excel y es fácil de parsear sin librerías.
- **Validar antes de simular**: si entra un proceso de 500K, nunca va a tener memoria y la simulación **no termina nunca**. Por eso se corta antes, con un mensaje claro.
- **Juntar todos los errores**: si el archivo tiene 3 errores, el usuario los ve todos de una vez, en lugar de corregir uno, volver a ejecutar y encontrarse con el siguiente.
- **Separar `parsear_csv` de `cargar_archivo`**: así se puede testear el parseo con un texto, sin depender de que exista un archivo.
- **Carga manual**: la consigna dice "cargar procesos por parte del usuario".

Reglas de validación

Regla	Motivo
1 a 10 procesos	Límite de la consigna
id entero, positivo y único	Si dos procesos tienen el mismo id, las tablas son ambiguas
1 ≤ tamaño ≤ 450	Es toda la memoria de usuario; si no, nunca entra
arribo ≥ 0	No existe el tiempo negativo
irrupción ≥ 1	Un proceso sin tiempo de CPU no tiene sentido

Cómo lo implementé

> *(completar al terminar: decisiones, problemas que aparecieron y cómo se resolvieron)*

Para exponer (1-2 min)

1. Mostrar el CSV de ejemplo y explicar cada columna.
2. Ejecutar con un archivo con errores → mostrar que lista todos los problemas.
3. Explicar por qué un proceso de más de 450K se rechaza (la simulación no terminaría).
4. Mostrar la carga manual.

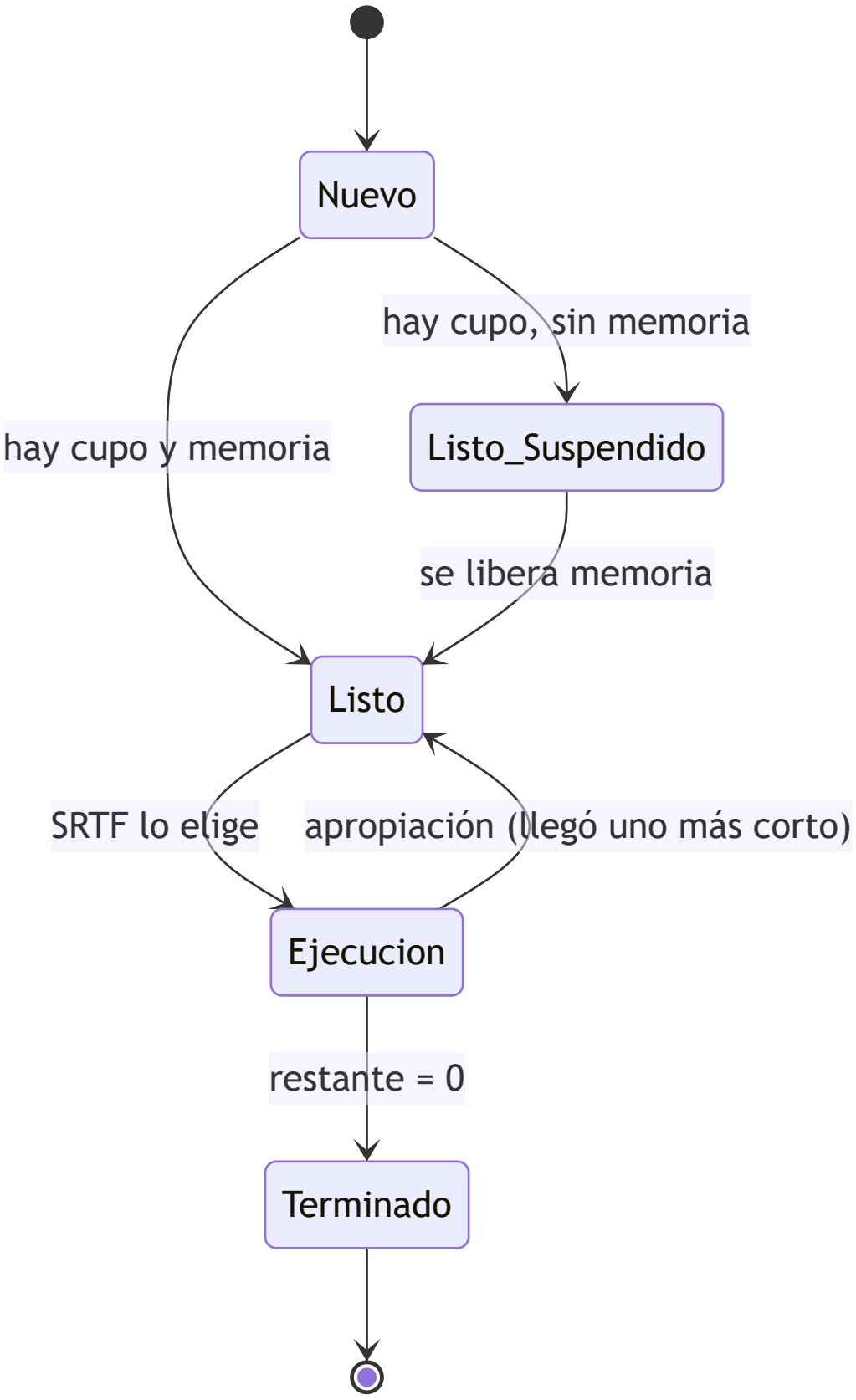
Preguntas probables: *¿Qué pasa si cargo 11 procesos?* · *¿Por qué no aceptás un tamaño de 500K si después se libera memoria?* (porque aunque toda la memoria quede libre son 450K, nunca alcanza).

M2 — Modelo de Proceso, estados y casos de prueba

Qué es

La clase `Proceso` funciona como el **PCB** (*Process Control Block*) del simulador: guarda todo lo que el SO sabe de un proceso (id, tamaño, arribo, irrupción, tiempo restante, estado, partición asignada e instante de fin).

Diagrama de estados



No existe *Bloqueado*: la consigna lo deja afuera para simplificar (no hay E/S).

Por qué se hizo así

- **Una sola clase compartida**: los 7 módulos hablan el mismo "idioma". Si cada uno armara su propio objeto, la integración sería un caos.

- **Validar transiciones**: un `Nuevo` no puede pasar directo a `Ejecución` (era el error #6 del diagrama). Si algún módulo se equivoca, el programa avisa en el momento en vez de dar resultados incorrectos sin que nadie lo note.
- `restante` **separado de** `irrupcion`: SRTF decide según lo que le *falta* al proceso, pero para calcular el tiempo de espera necesitamos la irrupción original.
- **TR y TE como getters**: se calculan a partir de datos que ya existen, así nunca quedan desactualizados.

Fórmulas

- **Tiempo de retorno** `TR = fin − arribo` → cuánto tardó desde que llegó hasta que terminó.
- **Tiempo de espera** `TE = TR − irrupción` → cuánto de ese tiempo estuvo esperando (sin usar CPU).

Casos de prueba a mano (tarea clave)

Los casos calculados a mano son la **prueba de que el simulador funciona bien**, y se muestran en el video. Para cada caso: tabla de memoria en cada evento, Gantt y TR/TE. El ejemplo resuelto está en `06-simulador.md`.

Cómo lo implementé

> *(completar)*

Para exponer (1-2 min)

1. Mostrar el diagrama de estados y explicar cada transición.
2. Explicar qué es el PCB y qué campos guardamos.
3. Mostrar uno de los casos resueltos a mano y que el simulador da lo mismo.

Preguntas probables: *¿Por qué no hay estado Bloqueado? · ¿Diferencia entre Listo y Listo/Suspendido?* (el suspendido fue admitido, pero no está en memoria principal).

M3 — Memoria con particiones variables (MVT) y Best-Fit

Qué es

Simula la memoria principal: **550K** en total, de los cuales **100K** son del SO (direcciones 0–99) y **450K** quedan para los procesos (direcciones 100–549).

Con **MVT** (*Multiprogramming with a Variable number of Tasks*) **no hay particiones fijas de antemano**: cuando entra un proceso se crea una partición **del tamaño exacto** que necesita.

```
Al inicio:        |  SO 100K  |              libre 450K                    |
Entra P1 (200K):  |  SO 100K  |  P1 200K  |          libre 250K            |
Entra P2 (150K):  |  SO 100K  |  P1 200K  |  P2 150K  |  libre 100K         |
Sale P1:          |  SO 100K  | libre 200K|  P2 150K  |  libre 100K         |  ← fragmentación
externa
```

Best-Fit

Entre todos los huecos libres que alcanzan, se elige **el más chico**. La idea es que el sobrante sea lo menor posible y que los huecos grandes queden disponibles para procesos grandes.

- Comparación: *First-Fit* toma el primero que alcanza (es más rápido); *Worst-Fit* toma el más grande.
- Desempate (decisión nuestra): si dos huecos miden lo mismo, se elige el de menor dirección.

Por qué se hizo así

- **Lista ordenada por dirección**: así es fácil mostrar la tabla y encontrar a los vecinos al liberar.
- **Fusionar al liberar** (*coalescing*): si quedan dos huecos de 100K pegados y no se fusionan, un proceso de 150K no entra aunque haya 200K libres contiguos. Era el error #8 del diagrama.
- **No compactamos**: la consigna no lo pide, y mover procesos en memoria tiene un costo. Lo explicamos como posible mejora.

Fragmentación (pregunta segura del coloquio)

- **Interna**: espacio desperdiciado *dentro* de una partición asignada. En MVT es **0**, porque la partición se crea del tamaño exacto. La columna se muestra porque la consigna la pide.
- **Externa**: memoria libre total suficiente, pero repartida en huecos que por separado no alcanzan. **Es el problema típico de MVT**, y se ve en la simulación cuando un proceso queda Listo/Suspendido aunque la suma de los huecos alcanzaría.

Cómo lo implementé

> *(completar)*

Para exponer (1-2 min)

1. Dibujar la memoria y mostrar una asignación (partir el hueco).
2. Mostrar un caso donde Best-Fit elige un hueco distinto al que elegiría First-Fit.
3. Mostrar una liberación con fusión de huecos.
4. Explicar por qué la fragmentación interna da 0 y señalar un caso de fragmentación externa.

M4 — Planificador de corto plazo SRTF + Gantt

Qué es

SRTF (*Shortest Remaining Time First*) decide **quién usa la CPU** en cada instante: siempre el proceso Listo al que le **falta menos** tiempo de ejecución.

Es **apropiativo** (*preemptive*): si llega un proceso con menos tiempo restante que el que está ejecutando, se lo saca de la CPU (vuelve a Listo **conservando su memoria**) y entra el más corto. Es la versión apropiativa de **SJF**.

Ejemplo (del caso `datos/procesos-ejemplo.csv`)

- t=0: llega P1 (irrupción 5) → ejecuta P1.
- t=1: llega P2 (irrupción 2). A P1 le quedan 4, y 2 < 4 → **apropiación**: P1 vuelve a Listo y entra P2.

```
Gantt:  | P1 | P2 | P2 | P4 | P1 | P1 | P1 | P1 | P3 | P3 | P3 |
        0    1    2    3    4    5    6    7    8    9    10   11
```

Por qué se hizo así

- **Apropiación sólo si es ESTRICTAMENTE menor**: si los tiempos empatan, cambiar de proceso no gana nada y sólo agrega un cambio de contexto. Era el error #7 del diagrama.
- **Desempates (menor arribo, después menor id)**: hacen que la simulación sea **determinista** (siempre da el mismo resultado) y que se pueda comparar con los casos resueltos a mano. Priorizar el que llegó antes es lo más justo.
- **Sólo se consideran los Listos**: un Listo/Suspendido no está en memoria, así que no puede ejecutar.
- **CPU ociosa**: si no hay Listos, no se ejecuta nada (error #5 del diagrama).
- **Gantt**: la consigna no lo pide, pero es la mejor forma de *mostrar* SRTF en el video.

Ventajas y desventajas (para el coloquio)

- ✅ Minimiza el tiempo de espera promedio.
- ❌ Puede generar **inanición** (*starvation*): si siguen llegando procesos cortos, uno largo puede no ejecutar nunca.
- ❌ En un SO real no se conoce de antemano la irrupción (hay que estimarla). En el simulador sí la conocemos porque viene en el archivo.

Cómo lo implementé

> *(completar)*

Para exponer (1-2 min)

1. Explicar SRTF y en qué se diferencia de SJF.
2. Mostrar en el Gantt la apropiación de t=1.
3. Explicar la regla de empate y por qué.
4. Mencionar la inanición.

M5 — Admisión: planificadores de largo y mediano plazo

Qué es

Antes de que un proceso pueda competir por la CPU, alguien tiene que decidir **si entra al sistema** y **si entra a memoria**. De eso se encarga este módulo.

Planificador	Qué decide	En el simulador
Largo plazo	Qué procesos nuevos se admiten	`admitir_nuevos`: Nuevo → Listo o Listo/Suspendido
Mediano plazo	Qué procesos suspendidos vuelven a memoria	`promover_suspendidos`: Listo/Susp → Listo
Corto plazo	Quién usa la CPU	SRTF (módulo M4)

Grado de multiprogramación = 5

La consigna dice que **Ejecución + Listo + Listo/Suspendido ≤ 5**. Los Nuevos que no entran **esperan** en estado Nuevo y se vuelven a intentar en cada tick.

```
Llega un proceso ─► ¿hay cupo (< 5)? ─No─► sigue en Nuevo (se reintenta)
                          │Sí
                          ▼
                   ¿Best-Fit encuentra hueco? ─No─► Listo/Suspendido (ocupa cupo, sin memoria)
                          │Sí
                          ▼
                       Listo
```

Por qué se hizo así

- `arribo <= t` **en vez de** `arribo == t`: con `==`, un proceso que no entró por falta de cupo se perdía para siempre (errores #2 y #3 del diagrama).
- **Primero suspendidos, después nuevos**: los suspendidos llegaron antes. Si los nuevos tuvieran prioridad, podrían quedarse con la memoria una y otra vez y el suspendido nunca entraría (error #4).
- **Orden por arribo**: es FIFO, lo más justo. El orden en el que vienen las filas del archivo no tiene que influir.
- **No bloquear la cola de suspendidos**: si el primero (300K) no entra pero el segundo (100K) sí, se sube el segundo. Así se aprovecha mejor la memoria. Es una **decisión de diseño** que hay que saber defender (la alternativa FIFO estricta es más justa, pero desperdicia memoria).

Cómo lo implementé

> *(completar)*

Para exponer (1-2 min)

1. Explicar los 3 niveles de planificación con la tabla.
2. Mostrar el caso del ejemplo: en t=2 P3 (300K) queda Listo/Suspendido porque sólo hay 100K libres.
3. Mostrar el caso con más de 5 procesos: el sexto espera en Nuevo.

M6 — Motor de simulación (reloj e integración)

Qué es

Es el "director de orquesta": maneja el reloj `t` y en cada tick llama a los demás módulos **en un orden fijo**. Por sí solo no decide nada de memoria ni de CPU: delega en los otros módulos.

El ciclo de un tick (el orden importa)

```
en el instante t:
  1. promoverSuspendidos     (mediano plazo — M5)
  2. admitirNuevos           (largo plazo — M5)
  3. planificar SRTF         (corto plazo — M4)
  4. si hubo arribo o fin  → mostrar estado y PAUSA (M7)
  5. ejecutar 1 unidad       (restante − 1; Gantt)
  6. si restante = 0       → Terminado, fin = t+1, liberar memoria (M3)
  7. t = t + 1
```

Por qué este orden: la memoria se reparte *antes* de que SRTF decida (así un proceso que acaba de llegar ya puede competir), y la pantalla se muestra *antes* de ejecutar (refleja quién va a usar la CPU en ese instante).

Separación motor / pantalla

`Simulador.paso()` **no imprime nada**: devuelve una `Instantanea` (objeto con todo el estado). `main.py` decide si la muestra y espera el Enter. Gracias a esto:

- se puede testear sin intervención (`tests/test_integracion.py`),
- después se puede reemplazar la consola por la interfaz con los estilos **sin tocar el motor**.

Ejemplo resuelto a mano (`datos/procesos-ejemplo.csv`)

Proceso	Tamaño	Arribo	Irrupción
P1	200K	0	5
P2	150K	1	2
P3	300K	2	3
P4	50K	3	1

t	Qué pasa	Memoria (usuario)	CPU
0	Llega P1 → Listo	P1[100-299] libre 250	P1
1	Llega P2 → Listo. 2 < 4 → **apropiación**	P1, P2[300-449], libre 100	P2
2	Llega P3 (300K): no entra → **Listo/Susp**	igual	P2
3	Termina P2 (libera y fusiona → libre 250). P3 sigue sin entrar. Llega P4 → Listo	P1, P4[300-349], libre 200	P4
4	Termina P4 (libre 250)	P1, libre 250	P1
8	Termina P1 → todo libre (450) → P3 sube a Listo	P3[100-399], libre 150	P3
11	Termina P3. Fin	todo libre	—

Proceso	Fin	TR = fin − arribo	TE = TR − irrupción
P1	8	8	3
P2	3	2	0
P3	11	9	6
P4	4	1	0
Promedio		**5,00**	**2,25**

Rendimiento = 4 procesos / 11 unidades = **0,36 procesos por unidad de tiempo**.

Cómo lo implementé

> *(completar)*

Para exponer (2-3 min, es la demo principal)

1. Explicar el ciclo de un tick y por qué en ese orden.
2. Ejecutar `python main.py datos\procesos-ejemplo.csv` (o el .exe instalado) y avanzar con Enter comentando cada evento contra la tabla de arriba.
3. Explicar la pausa: la consigna prohíbe corridas ininterrumpidas.
4. Mostrar todos los tests en verde: `python -m unittest discover -s tests -v`.

M7 — Salida, estadísticas y entrega

Qué es

Arma lo que ve el usuario: la pantalla de cada evento, el informe final y el paquete de entrega (ejecutable + HOWTO).

Lo que pide la consigna en cada evento (llegada o fin de un proceso)

1. **Estado del procesador**: qué proceso está ejecutando (u "ociosa").
2. **Tabla de particiones**: Id · Dirección de comienzo · Tamaño · Proceso asignado · Fragmentación interna.
3. **Colas** de Listos y de Listos/Suspendidos.

Ejemplo de cómo podría verse:

```
═══════ t = 2 │ Llegó: P3 ═══════
CPU: P2 (restante 1)

 Part │ Inicio │ Tamaño │ Proceso │ Frag.Int
──────┼────────┼────────┼─────────┼──────────
  SO  │    0   │  100K  │   SO    │    -
   2  │  100   │  200K  │   P1    │    0K
   3  │  300   │  150K  │   P2    │    0K
   4  │  450   │  100K  │  libre  │    -

Listos:           P1(4)
Listos/Susp.:     P3(3)
Esperando cupo:   —
```

Informe final

Por proceso: TR y TE. Además, los promedios y el **rendimiento** (*throughput*) = procesos terminados / tiempo total.

Por qué se hizo así

- **Las funciones devuelven texto y no hacen `print`**: así se pueden testear, y después se reemplaza la consola por la interfaz con los estilos sin tocar la lógica.
- **Las estadísticas van separadas del render**: el cálculo es lógica del dominio; mostrarlo es presentación.
- **Instalador**: el profe pidió que se lo llevemos con instalador, que lo instale y lo use, **sin instalar nada más y sin internet**. Se arma en dos pasos:
    1. **PyInstaller** mete Python + nuestro código en un solo `simulador.exe`. El profe **no necesita tener Python**.

2. **Inno Setup** (gratis) envuelve ese `.exe` en `SimuladorSetup.exe`: el clásico Siguiente → Siguiente → Instalar, con accesos directos en el menú Inicio y desinstalador.

Las dos herramientas sólo se instalan en la PC de quien **construye** el instalador, no en la del profe. Todo está en `instalador/construir.bat` y `instalador/simulador.iss`.

Cómo construir el instalador

```
pip install pyinstaller
(instalar Inno Setup 6 desde jrsoftware.org)
instalador\construir.bat      →  instalador\salida\SimuladorSetup.exe
```

Importante: probarlo en una PC que **no** tenga Python instalado antes de entregarlo. El antivirus a veces marca los `.exe` de PyInstaller: si pasa, hay que anotarlo en el HOWTO.

El HOWTO para el profe está en `HOWTO.md`, en la raíz del proyecto.

Cómo lo implementé

> *(completar)*

Para exponer (1-2 min)

1. Recorrer una pantalla de evento explicando cada parte.
2. Mostrar el informe final y explicar TR, TE y rendimiento con un proceso de ejemplo.
3. Mostrar la instalación con `SimuladorSetup.exe` en una PC sin Python.